

VVV On-Chain Intelligence Dashboard — Dune

Queries

Contract: 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf (VVV on Base)

Methodology: ERC-20 adaptation of Glassnode STH-NUPL / STH-SOPR

Dashboard Name: S2F Capital — VVV On-Chain Intelligence

Panel 1: New Entrant Cost Basis Distribution (STH-NUPL Core)

The foundational query. For every wallet that received VVV in the last 72 hours, compute their acquisition price at the block they received it, then calculate unrealized PnL vs current price.

```
-- PANEL 1A: New Entrant Acquisition Price & Unrealized PnL
-- Reconstructs Glassnode STH cost basis for ERC-20 on Base
-- Thresholds: EUPHORIA >40% gain for 80%+ wallets | CAPITULATION >20% loss for 7

WITH vvv_transfers AS (
    SELECT
        "to" AS wallet,
        block_time,
        block_number,
        amount / 1e18 AS vvv_amount,
        tx_hash
    FROM tokens.transfers
    WHERE blockchain = 'base'
        AND contract_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND block_time >= NOW() - INTERVAL '72' hour
        AND "to" != 0x0000000000000000000000000000000000000000 -- exclude burns
        AND amount > 0
),

-- First inbound transfer per wallet in the window = their "entry"
first_entry AS (
    SELECT
        wallet,
        MIN(block_time) AS entry_time,
        MIN(block_number) AS entry_block
    FROM vvv_transfers
```

```

        GROUP BY wallet
    ),

-- Match entry time to VVV price at that moment
entry_with_price AS (
    SELECT
        fe.wallet,
        fe.entry_time,
        p.price AS entry_price
    FROM first_entry fe
    LEFT JOIN prices.usd p
        ON p.blockchain = 'base'
        AND p.contract_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND p.minute = DATE_TRUNC('minute', fe.entry_time)
    ),

-- Get current VVV price (latest available)
current_price AS (
    SELECT price AS current_price
    FROM prices.usd
    WHERE blockchain = 'base'
        AND contract_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
    ORDER BY minute DESC
    LIMIT 1
    ),

-- Compute unrealized PnL per wallet
wallet_pnl AS (
    SELECT
        e.wallet,
        e.entry_time,
        e.entry_price,
        cp.current_price,
        (cp.current_price - e.entry_price) / NULLIF(e.entry_price, 0) AS unrealiz
    FROM entry_with_price e
    CROSS JOIN current_price cp
    WHERE e.entry_price > 0
    )

SELECT
    wallet,
    entry_time,
    entry_price,
    current_price,
    unrealized_pnl_pct,
    CASE
        WHEN unrealized_pnl_pct > 0.40 THEN 'EUPHORIA'

```

```

        WHEN unrealized_pnl_pct > 0.25 THEN 'GREED'
        WHEN unrealized_pnl_pct > 0.00 THEN 'HOPE'
        WHEN unrealized_pnl_pct > -0.20 THEN 'FEAR'
        ELSE 'CAPITULATION'
    END AS sentiment_phase
FROM wallet_pnl
ORDER BY unrealized_pnl_pct DESC

-- PANEL 1B: Aggregate STH-NUPL Gauge (single number + phase)
-- This is the headline metric for the dashboard

WITH vvv_transfers AS (
    SELECT
        "to" AS wallet,
        block_time,
        amount / 1e18 AS vvv_amount
    FROM tokens.transfers
    WHERE blockchain = 'base'
        AND contract_address = 0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND block_time >= NOW() - INTERVAL '72' hour
        AND "to" != 0x0000000000000000000000000000000000000000
        AND amount > 0
),

first_entry AS (
    SELECT wallet, MIN(block_time) AS entry_time
    FROM vvv_transfers
    GROUP BY wallet
),

entry_with_price AS (
    SELECT
        fe.wallet,
        p.price AS entry_price
    FROM first_entry fe
    LEFT JOIN prices.usd p
        ON p.blockchain = 'base'
        AND p.contract_address = 0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND p.minute = DATE_TRUNC('minute', fe.entry_time)
    WHERE p.price > 0
),

current_price AS (
    SELECT price AS current_price
    FROM prices.usd
    WHERE blockchain = 'base'

```

```

        AND contract_address = 0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
    ORDER BY minute DESC
    LIMIT 1
)

SELECT
    COUNT(*) AS total_new_wallets,
    AVG((cp.current_price - e.entry_price) / e.entry_price) AS mean_unrealized_pn
    APPROX_PERCENTILE((cp.current_price - e.entry_price) / e.entry_price, 0.5) AS
    COUNT(*) FILTER (WHERE (cp.current_price - e.entry_price) / e.entry_price > 0
    COUNT(*) FILTER (WHERE (cp.current_price - e.entry_price) / e.entry_price > 0
    COUNT(*) FILTER (WHERE (cp.current_price - e.entry_price) / e.entry_price < -
    CASE
        WHEN COUNT(*) FILTER (WHERE (cp.current_price - e.entry_price) / e.entry_
        WHEN COUNT(*) FILTER (WHERE (cp.current_price - e.entry_price) / e.entry_
        WHEN COUNT(*) FILTER (WHERE (cp.current_price - e.entry_price) / e.entry_
        WHEN COUNT(*) FILTER (WHERE (cp.current_price - e.entry_price) / e.entry_
        ELSE 'NEUTRAL'
    END AS regime_signal
FROM entry_with_price e
CROSS JOIN current_price cp

```

```

-- PANEL 1C: STH-NUPL Time Series (daily rolling 72hr window)
-- Track how new entrant sentiment evolves over time
-- Visualization: line chart with colored phase bands

```

```

WITH date_series AS (
    SELECT dt
    FROM UNNEST(SEQUENCE(
        CURRENT_DATE - INTERVAL '30' day,
        CURRENT_DATE,
        INTERVAL '1' day
    )) AS t(dt)
),

```

```

vvv_transfers AS (
    SELECT
        "to" AS wallet,
        block_time,
        DATE_TRUNC('day', block_time) AS transfer_day,
        amount / 1e18 AS vvv_amount
    FROM tokens.transfers
    WHERE blockchain = 'base'
        AND contract_address = 0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND block_time >= CURRENT_DATE - INTERVAL '33' day -- 30 days + 3 day look
        AND "to" != 0x0000000000000000000000000000000000000000

```

```

        AND amount > 0
    ),

daily_prices AS (
    SELECT
        DATE_TRUNC('day', minute) AS price_day,
        AVG(price) AS avg_price
    FROM prices.usd
    WHERE blockchain = 'base'
        AND contract_address = 0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND minute >= CURRENT_DATE - INTERVAL '33' day
    GROUP BY 1
),

-- For each day, find wallets that entered in the prior 72hr window
daily_entrants AS (
    SELECT
        ds.dt AS snapshot_date,
        t.wallet,
        MIN(t.block_time) AS entry_time
    FROM date_series ds
    JOIN vvv_transfers t
        ON t.block_time >= ds.dt - INTERVAL '3' day
        AND t.block_time < ds.dt + INTERVAL '1' day
    GROUP BY ds.dt, t.wallet
),

daily_pnl AS (
    SELECT
        de.snapshot_date,
        de.wallet,
        ep.avg_price AS entry_price,
        sp.avg_price AS snapshot_price,
        (sp.avg_price - ep.avg_price) / NULLIF(ep.avg_price, 0) AS pnl
    FROM daily_entrants de
    LEFT JOIN daily_prices ep ON ep.price_day = DATE_TRUNC('day', de.entry_time)
    LEFT JOIN daily_prices sp ON sp.price_day = de.snapshot_date
    WHERE ep.avg_price > 0
)

SELECT
    snapshot_date,
    COUNT(*) AS new_wallets,
    AVG(pnl) AS mean_sth_nupl,
    APPROX_PERCENTILE(pnl, 0.5) AS median_sth_nupl,
    COUNT(*) FILTER (WHERE pnl > 0) * 1.0 / NULLIF(COUNT(*), 0) AS pct_in_profit,
    COUNT(*) FILTER (WHERE pnl > 0.40) * 1.0 / NULLIF(COUNT(*), 0) AS pct_euphori

```

```
FROM daily_pnl
GROUP BY snapshot_date
ORDER BY snapshot_date
```

Panel 2: CEX Netflow Tracker

Track VVV flowing to/from known exchange addresses. This replicates the netflow analysis Roy showed in his tweet.

```
-- PANEL 2A: Daily CEX Netflows
-- Negative netflow = tokens leaving exchanges (bullish accumulation signal)
-- Uses Dune's labels.addresses for CEX identification
```

```
WITH cex_addresses AS (
    SELECT DISTINCT address
    FROM labels.addresses
    WHERE blockchain = 'base'
        AND category = 'cex'
),

inflows AS (
    -- Tokens going TO exchanges (potential sell pressure)
    SELECT
        DATE_TRUNC('day', t.block_time) AS day,
        SUM(t.amount / 1e18) AS tokens_in,
        COUNT(DISTINCT t."from") AS unique_depositors
    FROM tokens.transfers t
    JOIN cex_addresses cex ON t."to" = cex.address
    WHERE t.blockchain = 'base'
        AND t.contract_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND t.block_time >= NOW() - INTERVAL '30' day
    GROUP BY 1
),
```

```
outflows AS (
    -- Tokens leaving exchanges (accumulation / self-custody)
    SELECT
        DATE_TRUNC('day', t.block_time) AS day,
        SUM(t.amount / 1e18) AS tokens_out,
        COUNT(DISTINCT t."to") AS unique_withdrawers
    FROM tokens.transfers t
    JOIN cex_addresses cex ON t."from" = cex.address
    WHERE t.blockchain = 'base'
        AND t.contract_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
```

```

        AND t.block_time >= NOW() - INTERVAL '30' day
    GROUP BY 1
),

daily_prices AS (
    SELECT
        DATE_TRUNC('day', minute) AS day,
        AVG(price) AS avg_price
    FROM prices.usd
    WHERE blockchain = 'base'
        AND contract_address = 0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND minute >= NOW() - INTERVAL '30' day
    GROUP BY 1
)

SELECT
    COALESCE(i.day, o.day) AS day,
    COALESCE(i.tokens_in, 0) AS cex_inflow_tokens,
    COALESCE(o.tokens_out, 0) AS cex_outflow_tokens,
    COALESCE(o.tokens_out, 0) - COALESCE(i.tokens_in, 0) AS net_flow_tokens,
    (COALESCE(o.tokens_out, 0) - COALESCE(i.tokens_in, 0)) * p.avg_price AS net_f
    COALESCE(i.unique_depositors, 0) AS depositors,
    COALESCE(o.unique_withdrawers, 0) AS withdrawers,
    p.avg_price AS vvv_price
FROM inflows i
FULL OUTER JOIN outflows o ON i.day = o.day
LEFT JOIN daily_prices p ON COALESCE(i.day, o.day) = p.day
ORDER BY day DESC

-- PANEL 2B: Cumulative CEX Netflow (running total)
-- Shows whether exchanges are accumulating or draining over time
-- Visualization: area chart - below zero = bullish

WITH cex_addresses AS (
    SELECT DISTINCT address
    FROM labels.addresses
    WHERE blockchain = 'base'
        AND category = 'cex'
),

daily_flows AS (
    SELECT
        DATE_TRUNC('day', t.block_time) AS day,
        SUM(CASE WHEN cex_to.address IS NOT NULL THEN t.amount / 1e18 ELSE 0 END)
        SUM(CASE WHEN cex_from.address IS NOT NULL THEN t.amount / 1e18 ELSE 0 EN
    FROM tokens.transfers t

```

```

LEFT JOIN cex_addresses cex_to ON t."to" = cex_to.address
LEFT JOIN cex_addresses cex_from ON t."from" = cex_from.address
WHERE t.blockchain = 'base'
      AND t.contract_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
      AND t.block_time >= NOW() - INTERVAL '90' day
      AND (cex_to.address IS NOT NULL OR cex_from.address IS NOT NULL)
GROUP BY 1
)

SELECT
    day,
    outflow - inflow AS daily_net,
    SUM(outflow - inflow) OVER (ORDER BY day) AS cumulative_net_flow
FROM daily_flows
ORDER BY day

```

Panel 3: Holder Vintage Bands

What percentage of VVV supply is held by wallets of different ages? Shows whether the holder base is maturing or dominated by hot money.

```

-- PANEL 3: Holder Cohort Analysis by Wallet Age
-- Classifies current holders by when they first received VVV
-- Visualization: stacked area chart

WITH all_receipts AS (
    SELECT
        "to" AS wallet,
        MIN(block_time) AS first_received
    FROM tokens.transfers
    WHERE blockchain = 'base'
          AND contract_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
          AND "to" != 0x0000000000000000000000000000000000000000
          AND amount > 0
    GROUP BY "to"
),

current_balances AS (
    SELECT
        address AS wallet,
        balance / 1e18 AS vvv_balance
    FROM balances.erc20_latest
    WHERE blockchain = 'base'
          AND token_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf

```

```

        AND balance > 0
    ),

holder_cohorts AS (
    SELECT
        cb.wallet,
        cb.vvv_balance,
        ar.first_received,
        CASE
            WHEN ar.first_received >= NOW() - INTERVAL '24' hour THEN '< 24h'
            WHEN ar.first_received >= NOW() - INTERVAL '7' day THEN '1-7d'
            WHEN ar.first_received >= NOW() - INTERVAL '30' day THEN '7-30d'
            WHEN ar.first_received >= NOW() - INTERVAL '90' day THEN '30-90d'
            ELSE '90d+'
        END AS cohort
    FROM current_balances cb
    LEFT JOIN all_receipts ar ON cb.wallet = ar.wallet
)

SELECT
    cohort,
    COUNT(*) AS num_wallets,
    SUM(vvv_balance) AS total_vvv,
    SUM(vvv_balance) / NULLIF((SELECT SUM(vvv_balance) FROM holder_cohorts), 0) AS
    avg(vvv_balance) AS avg_balance,
    APPROX_PERCENTILE(vvv_balance, 0.5) AS median_balance
FROM holder_cohorts
GROUP BY cohort
ORDER BY
    CASE cohort
        WHEN '< 24h' THEN 1
        WHEN '1-7d' THEN 2
        WHEN '7-30d' THEN 3
        WHEN '30-90d' THEN 4
        WHEN '90d+' THEN 5
    END

```

Panel 4: sVVV Staking Flow (Conviction Proxy)

Net staking vs unstaking activity. Rising staking = holders locking tokens for API inference access = structural demand removal from circulating supply.

```

-- PANEL 4: sVVV Staking Flows
-- sVVV contract: look up the staking contract on BaseScan

```

```
-- Track VVV transfers TO the staking contract (stakes) vs FROM (unstakes)
-- NOTE: Update STAKING_CONTRACT with actual sVVV staking address
```

```
WITH staking_flows AS (
    SELECT
        DATE_TRUNC('day', block_time) AS day,
        SUM(CASE
            WHEN "to" = 0x__STAKING_CONTRACT_ADDRESS__ THEN amount / 1e18
            ELSE 0
        END) AS tokens_staked,
        SUM(CASE
            WHEN "from" = 0x__STAKING_CONTRACT_ADDRESS__ THEN amount / 1e18
            ELSE 0
        END) AS tokens_unstaked,
        COUNT(DISTINCT CASE
            WHEN "to" = 0x__STAKING_CONTRACT_ADDRESS__ THEN "from"
        END) AS unique_stakers,
        COUNT(DISTINCT CASE
            WHEN "from" = 0x__STAKING_CONTRACT_ADDRESS__ THEN "to"
        END) AS unique_unstakers
    FROM tokens.transfers
    WHERE blockchain = 'base'
        AND contract_address = 0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND block_time >= NOW() - INTERVAL '90' day
        AND (
            "to" = 0x__STAKING_CONTRACT_ADDRESS__
            OR "from" = 0x__STAKING_CONTRACT_ADDRESS__
        )
    GROUP BY 1
),
```

```
daily_prices AS (
    SELECT
        DATE_TRUNC('day', minute) AS day,
        AVG(price) AS avg_price
    FROM prices.usd
    WHERE blockchain = 'base'
        AND contract_address = 0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND minute >= NOW() - INTERVAL '90' day
    GROUP BY 1
)
```

```
SELECT
    sf.day,
    sf.tokens_staked,
    sf.tokens_unstaked,
    sf.tokens_staked - sf.tokens_unstaked AS net_staking,
```

```

SUM(sf.tokens_staked - sf.tokens_unstaked) OVER (ORDER BY sf.day) AS cumulati
(sf.tokens_staked - sf.tokens_unstaked) * p.avg_price AS net_staking_usd,
sf.unique_stakers,
sf.unique_unstakers
FROM staking_flows sf
LEFT JOIN daily_prices p ON sf.day = p.day
ORDER BY sf.day DESC

```

Panel 5: Whale Wallet Monitor

Top 20 non-exchange, non-contract wallets with balance changes over the last 7 days.

```

-- PANEL 5: Top Holder Balance Changes (7-day delta)
-- Identifies accumulation or distribution by large wallets

WITH current_balances AS (
    SELECT
        address AS wallet,
        balance / 1e18 AS current_balance
    FROM balances.erc20_latest
    WHERE blockchain = 'base'
        AND token_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND balance / 1e18 > 1000 -- minimum 1K VVV to qualify as whale
),

-- Net transfers in the last 7 days per wallet
recent_net_transfers AS (
    SELECT
        wallet,
        SUM(net_amount) AS seven_day_change
    FROM (
        -- Inflows
        SELECT "to" AS wallet, SUM(amount / 1e18) AS net_amount
        FROM tokens.transfers
        WHERE blockchain = 'base'
            AND contract_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
            AND block_time >= NOW() - INTERVAL '7' day
        GROUP BY "to"

        UNION ALL

        -- Outflows (negative)
        SELECT "from" AS wallet, -SUM(amount / 1e18) AS net_amount
        FROM tokens.transfers
    )

```

```

        WHERE blockchain = 'base'
        AND contract_address = 0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND block_time >= NOW() - INTERVAL '7' day
        GROUP BY "from"
    ) sub
GROUP BY wallet
),

-- Exclude known contracts/exchanges
cex_addresses AS (
    SELECT DISTINCT address
    FROM labels.addresses
    WHERE blockchain = 'base'
    AND category = 'cex'
)

SELECT
    cb.wallet,
    COALESCE(l.name, CAST(cb.wallet AS VARCHAR)) AS label,
    cb.current_balance,
    COALESCE(rnt.seven_day_change, 0) AS seven_day_change,
    COALESCE(rnt.seven_day_change, 0) / NULLIF(cb.current_balance - COALESCE(rnt.
CASE
    WHEN COALESCE(rnt.seven_day_change, 0) > 0 THEN 'ACCUMULATING'
    WHEN COALESCE(rnt.seven_day_change, 0) < 0 THEN 'DISTRIBUTING'
    ELSE 'FLAT'
END AS behavior
FROM current_balances cb
LEFT JOIN recent_net_transfers rnt ON cb.wallet = rnt.wallet
LEFT JOIN labels.addresses l
    ON cb.wallet = l.address AND l.blockchain = 'base'
LEFT JOIN cex_addresses cex ON cb.wallet = cex.address
WHERE cex.address IS NULL -- exclude CEX wallets
    AND cb.wallet != 0x0000000000000000000000000000000000000000000000000000000000000000
ORDER BY cb.current_balance DESC
LIMIT 30

```

Panel 6: DIEM Minting Activity (Venice Ecosystem Health)

DIEM is minted by locking sVVV. Surging DIEM mints = protocols/agents ramping up API consumption through Venice.

```

-- PANEL 6: DIEM Token Activity
-- DIEM contract on Base - update address below

```

```

-- Minting (from 0x0) = new DIEM created by locking sVVV
-- Burns (to 0x0) = DIEM redeemed

WITH diem_activity AS (
    SELECT
        DATE_TRUNC('day', block_time) AS day,
        SUM(CASE
            WHEN "from" = 0x0000000000000000000000000000000000000000000000000000000000000000
            THEN amount / 1e18 ELSE 0
        END) AS diem_minted,
        SUM(CASE
            WHEN "to" = 0x0000000000000000000000000000000000000000000000000000000000000000
            THEN amount / 1e18 ELSE 0
        END) AS diem_burned,
        COUNT(DISTINCT CASE
            WHEN "from" = 0x0000000000000000000000000000000000000000000000000000000000000000
            THEN "to"
        END) AS unique_minters
    FROM tokens.transfers
    WHERE blockchain = 'base'
        AND contract_address = 0x__DIEM_CONTRACT_ADDRESS__
        AND block_time >= NOW() - INTERVAL '90' day
    GROUP BY 1
)

SELECT
    day,
    diem_minted,
    diem_burned,
    diem_minted - diem_burned AS net_diem,
    SUM(diem_minted - diem_burned) OVER (ORDER BY day) AS cumulative_diem_supply,
    unique_minters
FROM diem_activity
ORDER BY day DESC

```

Panel 7: Buy/Sell Pressure Ratio (DEX Trade Direction)

Classifies DEX swaps as buys vs sells based on which side of the pair VVV is on.

```

-- PANEL 7: DEX Buy/Sell Ratio
-- Uses dex.trades to classify trade direction
-- Buy = VVV is token_bought | Sell = VVV is token_sold

WITH vvv_dex_trades AS (

```

```

SELECT
    DATE_TRUNC('day', block_time) AS day,
    CASE
        WHEN token_bought_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21b
        WHEN token_sold_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
    END AS direction,
    amount_usd,
    taker
FROM dex.trades
WHERE blockchain = 'base'
    AND (
        token_bought_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        OR token_sold_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
    )
    AND block_time >= NOW() - INTERVAL '30' day
)

SELECT
    day,
    COUNT(*) FILTER (WHERE direction = 'BUY') AS buy_count,
    COUNT(*) FILTER (WHERE direction = 'SELL') AS sell_count,
    SUM(amount_usd) FILTER (WHERE direction = 'BUY') AS buy_volume_usd,
    SUM(amount_usd) FILTER (WHERE direction = 'SELL') AS sell_volume_usd,
    COUNT(DISTINCT taker) FILTER (WHERE direction = 'BUY') AS unique_buyers,
    COUNT(DISTINCT taker) FILTER (WHERE direction = 'SELL') AS unique_sellers,
    COALESCE(
        SUM(amount_usd) FILTER (WHERE direction = 'BUY')
        / NULLIF(SUM(amount_usd) FILTER (WHERE direction = 'SELL'), 0),
        0
    ) AS buy_sell_ratio
FROM vvv_dex_trades
GROUP BY day
ORDER BY day DESC

```

Panel 8: Transfer Volume vs Price (Divergence Detection)

When transfer volume spikes without price following (or vice versa), it signals distribution or stealth accumulation.

```

-- PANEL 8: Transfer Volume vs Price Overlay
-- Visualization: dual-axis chart (volume bars + price line)

SELECT
    DATE_TRUNC('day', t.block_time) AS day,

```

```

COUNT(*) AS num_transfers,
COUNT(DISTINCT t."from") + COUNT(DISTINCT t."to") AS active_addresses,
SUM(t.amount / 1e18) AS total_vvv_transferred,
SUM(t.amount / 1e18) * p.avg_price AS volume_usd,
p.avg_price AS vvv_price
FROM tokens.transfers t
LEFT JOIN (
    SELECT
        DATE_TRUNC('day', minute) AS day,
        AVG(price) AS avg_price
    FROM prices.usd
    WHERE blockchain = 'base'
        AND contract_address = 0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND minute >= NOW() - INTERVAL '90' day
    GROUP BY 1
) p ON DATE_TRUNC('day', t.block_time) = p.day
WHERE t.blockchain = 'base'
    AND t.contract_address = 0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
    AND t.block_time >= NOW() - INTERVAL '90' day
GROUP BY DATE_TRUNC('day', t.block_time), p.avg_price
ORDER BY day DESC

```

Panel 9: OpenClaw Migration Tracker — New Staker Acceleration

The critical migration signal. First-time VVV stakers spiking after April 4, 2026 (Anthropic's OpenClaw ban date) = displaced agent operators locking in Venice compute. This is the single most important panel for proving the migration thesis.

```

-- PANEL 9A: Daily First-Time VVV Stakers
-- A spike post-April 4 2026 = OpenClaw migration signal
-- Visualization: bar chart with April 4 vertical annotation line

WITH staking_events AS (
    SELECT
        "from" AS staker,
        block_time,
        amount / 1e18 AS vvv_staked
    FROM tokens.transfers
    WHERE blockchain = 'base'
        AND contract_address = 0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND "to" = 0x__STAKING_CONTRACT_ADDRESS__
        AND block_time >= DATE '2026-03-01'

```

```
),
```

```
first_stakes AS (  
    SELECT  
        staker,  
        MIN(block_time) AS first_stake_time,  
        MIN(vvv_staked) AS first_stake_amount  
    FROM staking_events  
    GROUP BY staker  
)
```

```
SELECT  
    DATE_TRUNC('day', first_stake_time) AS day,  
    COUNT(*) AS new_stakers,  
    SUM(first_stake_amount) AS total_first_stake_vvv,  
    AVG(first_stake_amount) AS avg_first_stake_size,  
    SUM(COUNT(*)) OVER (ORDER BY DATE_TRUNC('day', first_stake_time)) AS cumulati  
CASE  
    WHEN DATE_TRUNC('day', first_stake_time) >= DATE '2026-04-04' THEN 'POST_  
    ELSE 'PRE_BAN'  
END AS period  
FROM first_stakes  
GROUP BY 1  
ORDER BY 1
```

```
-- PANEL 9B: Pre-Ban vs Post-Ban Staking Comparison  
-- Summary counter showing acceleration factor  
-- Visualization: two big numbers side by side
```

```
WITH staking_events AS (  
    SELECT  
        "from" AS staker,  
        block_time  
    FROM tokens.transfers  
    WHERE blockchain = 'base'  
        AND contract_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf  
        AND "to" = 0x__STAKING_CONTRACT_ADDRESS__  
        AND block_time >= DATE '2026-03-01'  
)
```

```
first_stakes AS (  
    SELECT staker, MIN(block_time) AS first_stake_time  
    FROM staking_events  
    GROUP BY staker  
)
```

```

periods AS (
    SELECT
        CASE
            WHEN first_stake_time >= TIMESTAMP '2026-04-04 19:00:00' -- 12pm PT
            THEN 'POST_BAN'
            ELSE 'PRE_BAN'
        END AS period,
        staker,
        first_stake_time
    FROM first_stakes
    WHERE first_stake_time >= DATE '2026-03-04' -- 30 days pre-ban window
)

SELECT
    period,
    COUNT(*) AS new_stakers,
    COUNT(*) * 1.0 / DATE_DIFF('day',
        MIN(first_stake_time),
        MAX(first_stake_time)
    ) AS stakers_per_day,
    MIN(first_stake_time) AS period_start,
    MAX(first_stake_time) AS period_end
FROM periods
GROUP BY period

```

Panel 10: DIEM Mint Acceleration (Compute Lockup Velocity)

DIEM minting is the strongest conversion signal — it means someone is locking sVVV specifically to secure perpetual API compute access. A mint acceleration ratio > 2x post-ban is notable, > 5x is a migration wave.

```

-- PANEL 10A: DIEM Mint Acceleration Ratio
-- Compares each day's minting vs trailing 7-day average
-- Visualization: bar chart with acceleration ratio line overlay

```

```

WITH daily_mints AS (
    SELECT
        DATE_TRUNC('day', block_time) AS day,
        SUM(amount / 1e18) AS diem_minted,
        COUNT(DISTINCT "to") AS unique_minters
    FROM tokens.transfers
    WHERE blockchain = 'base'
        AND contract_address = 0x__DIEM_CONTRACT_ADDRESS__
        AND "from" = 0x0000000000000000000000000000000000000000

```

```

        AND block_time >= DATE '2026-03-01'
    GROUP BY 1
)

SELECT
    day,
    diem_minted,
    unique_minters,
    AVG(diem_minted) OVER (
        ORDER BY day ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING
    ) AS seven_day_avg_prior,
    diem_minted / NULLIF(
        AVG(diem_minted) OVER (
            ORDER BY day ROWS BETWEEN 7 PRECEDING AND 1 PRECEDING
        ), 0
    ) AS mint_acceleration_ratio,
    CASE
        WHEN day >= DATE '2026-04-04' THEN 'POST_BAN'
        ELSE 'PRE_BAN'
    END AS period
FROM daily_mints
ORDER BY day

-- PANEL 10B: New DIEM Minter Wallets (first-time minters)
-- Wallets minting DIEM for the first time = new compute consumers
-- Cross-reference with Panel 9 to see staker-minter conversion funnel

WITH mint_events AS (
    SELECT
        "to" AS minter,
        block_time,
        amount / 1e18 AS diem_amount
    FROM tokens.transfers
    WHERE blockchain = 'base'
        AND contract_address = 0x__DIEM_CONTRACT_ADDRESS__
        AND "from" = 0x0000000000000000000000000000000000000000000000000000000000000000
        AND block_time >= DATE '2026-03-01'
),

first_mints AS (
    SELECT
        minter,
        MIN(block_time) AS first_mint_time,
        SUM(diem_amount) AS total_diem_minted
    FROM mint_events
    GROUP BY minter

```

)

SELECT

```
    DATE_TRUNC('day', first_mint_time) AS day,
    COUNT(*) AS new_minters,
    SUM(total_diem_minted) AS total_diem_by_new_minters,
    AVG(total_diem_minted) AS avg_diem_per_new_minter,
    SUM(COUNT(*)) OVER (ORDER BY DATE_TRUNC('day', first_mint_time)) AS cumulative
CASE
    WHEN DATE_TRUNC('day', first_mint_time) >= DATE '2026-04-04' THEN 'POST_B
    ELSE 'PRE_BAN'
END AS period
```

FROM first_mints

GROUP BY 1

ORDER BY 1

-- PANEL 10C: Venice API Proxy Signal - VVV→sVVV→DIEM Conversion Funnel
-- Tracks the full journey: acquire VVV → stake → mint DIEM → consume compute
-- Visualization: funnel chart or stacked bar

WITH vvv_buyers AS (

```
    SELECT COUNT(DISTINCT taker) AS unique_buyers
    FROM dex.trades
    WHERE blockchain = 'base'
        AND token_bought_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND block_time >= DATE '2026-04-04'
```

),

vvv_stakers AS (

```
    SELECT COUNT(DISTINCT "from") AS unique_stakers
    FROM tokens.transfers
    WHERE blockchain = 'base'
        AND contract_address = 0xacFE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf
        AND "to" = 0x__STAKING_CONTRACT_ADDRESS__
        AND block_time >= DATE '2026-04-04'
```

),

diem_minters AS (

```
    SELECT COUNT(DISTINCT "to") AS unique_minters
    FROM tokens.transfers
    WHERE blockchain = 'base'
        AND contract_address = 0x__DIEM_CONTRACT_ADDRESS__
        AND "from" = 0x0000000000000000000000000000000000000000000000000000000000000000
        AND block_time >= DATE '2026-04-04'
```

)

```

SELECT
    'VVV Buyers (DEX)' AS stage, unique_buyers AS count FROM vvv_buyers
UNION ALL
SELECT
    'VVV Stakers' AS stage, unique_stakers AS count FROM vvv_stakers
UNION ALL
SELECT
    'DIEM Minters' AS stage, unique_minters AS count FROM diem_minters

```

Setup Notes

Contract Addresses to Fill In

Token	Address	Notes
VVV	0xacfE6019Ed1A7Dc6f7B508C02d1b04ec88cC21bf	Confirmed
sVVV	Look up on BaseScan	Staking contract — check Venice docs
DIEM	Look up on BaseScan	Minted by locking sVVV

Dashboard Layout (Recommended Order)

Section A: Market Sentiment (STH-NUPL)

1. **STH-NUPL Gauge** (Panel 1B) — single counter, top of dashboard
2. **STH-NUPL Time Series** (Panel 1C) — line chart with phase bands
3. **Cost Basis Distribution** (Panel 1A) — histogram

Section B: Flow Analysis

4. **CEX Netflows** (Panel 2A + 2B) — bar chart + cumulative area
5. **Buy/Sell Ratio** (Panel 7) — bar chart with ratio overlay
6. **Volume vs Price** (Panel 8) — dual-axis

Section C: Holder Structure

7. **Holder Vintage Bands** (Panel 3) — stacked area
8. **Whale Monitor** (Panel 5) — table

Section D: Venice Ecosystem / OpenClaw Migration

9. **New Staker Acceleration** (Panel 9A) — bar chart with April 4 annotation
10. **Pre vs Post Ban Comparison** (Panel 9B) — counter cards
11. **DIEM Mint Acceleration** (Panel 10A) — bar + ratio line
12. **New DIEM Minters** (Panel 10B) — bar chart
13. **Conversion Funnel** (Panel 10C) — funnel chart
- 14.

Staking Flows (Panel 4) — bar chart with cumulative line 15. **DIEM Activity** (Panel 6) — bar chart

Key Thresholds (from FAI framework, calibrated for high-beta tokens)

Signal	Condition	Interpretation
EUPHORIA	80%+ of 72hr wallets at >40% gain	Highest pullback probability
GREED	60%+ at >25% gain	Correction pressure building
FEAR	50%+ underwater	Weak hand selling pressure
CAPITULATION	70%+ at >20% loss	Contrarian bottom signal
CEX drain	Cumulative netflow negative + accelerating	Bullish accumulation
Staking surge	Net staking > 2x 30-day avg	Structural demand signal

Migration-Specific Thresholds (OpenClaw → Venice)

Signal	Condition	Interpretation
Migration wave	DIEM mint acceleration ratio > 5x post-April 4	Strong agent migration evidence
Migration notable	DIEM mint acceleration ratio > 2x post-April 4	Early migration signal
New staker spike	Daily first-time stakers > 3x pre-ban avg	Compute demand influx
Funnel conversion	>30% of post-ban VVV buyers also stake within 7 days	Usage-driven demand (not speculative)
DIEM minter quality	Avg DIEM per new minter > 1.0	Serious compute consumers (\$1+/day)

Key Date: April 4, 2026 19:00 UTC (12pm PT)

This is the Anthropic OpenClaw ban enforcement timestamp. All migration panels use this as the dividing line between PRE_BAN and POST_BAN periods. Panels 9 and 10 should

include a vertical annotation line at this date.

Dune-Specific Tips

- Always include `blockchain = 'base'` and `block_date` or `block_time` filters first for query performance
- `prices.usd` may have gaps for VVV — use `prices.day` as fallback for daily granularity
- `labels.addresses` coverage on Base may be incomplete for CEX hot wallets — supplement with known Coinbase addresses manually if needed
- `balances.erc20_latest` is derived from cumulative transfers, may lag slightly
- `dex.trades` on Base covers Aerodrome, Uniswap V3, and other Base DEXs